

Programming Lab – All Programs

LEX | YACC | SHELL | Python

■■■ LEX Programs ■■■

1) Valid Integer Constant

```
%{
#include<stdio.h>
%}
%%
[0-9]+  { printf("Valid Integer: %s\n", yytext); }
.      { printf("Invalid\n"); }
%%
int main(){ yylex(); return 0; }
int yywrap(){ return 1; }
```

2) Valid Identifier

```
%{
#include<stdio.h>
%}
%%
[a-zA-Z_][a-zA-Z0-9_]* { printf("Valid Identifier: %s\n", yytext); }
.                      { printf("Invalid\n"); }
%%
int main(){ yylex(); return 0; }
int yywrap(){ return 1; }
```

3) Valid Float Constant

```
%{
#include<stdio.h>
%}
%%
[0-9]+ "." [0-9]+  { printf("Valid Float: %s\n", yytext); }
.                  { printf("Invalid\n"); }
%%
int main(){ yylex(); return 0; }
int yywrap(){ return 1; }
```

■■■ YACC Program ■■■

4) YACC – Calculator Program

```
/* calc.l */
%{
#include "y.tab.h"
%}
%%
[0-9]+ { yylval=atoi(yytext); return NUM; }
[+\-*/] { return yytext[0]; }
\n      { return '\n'; }
.        ;
%%
int yywrap(){ return 1; }

/* calc.y */
%{
#include<stdio.h>
%}
%token NUM
%%
expr: expr '+' expr { printf("%d\n", $1+$3); }
    | expr '-' expr { printf("%d\n", $1-$3); }
    | expr '*' expr { printf("%d\n", $1*$3); }
    | expr '/' expr { printf("%d\n", $1/$3); }
    | NUM           { $$=$1; }
    ;
%%
int main(){ yyparse(); return 0; }
int yyerror(char *s){ printf("Error: %s\n",s); return 0; }
```

5) YACC – Arithmetic Expression (+ - *)

```
/* Same as above but restricted to +, -, * operators.
   Modify the grammar rule to remove '/' if needed. */
/* Compile:
   lex calc.l && yacc -d calc.y
   gcc lex.yy.c y.tab.c -o calc -ll
   echo "3+4*2" | ./calc
*/
```

■■■ SHELL Programs ■■■

6) Hello World

```
#!/bin/bash
echo "Hello World"
```

7) Add 2 Numbers

```
#!/bin/bash
read -p "Enter first number: " a
read -p "Enter second number: " b
echo "Sum = $((a + b))"
```

8) Average of 5 Numbers

```
#!/bin/bash
sum=0
for i in 1 2 3 4 5; do
    read -p "Enter number $i: " n
    sum=$((sum + n))
done
echo "Average = $(echo "scale=2; $sum/5" | bc)"
```

9) Greater of 2 Numbers

```
#!/bin/bash
read -p "Enter a: " a
read -p "Enter b: " b
if [ $a -gt $b ]; then
    echo "$a is greater"
else
    echo "$b is greater"
fi
```

10) Greatest of 3 Numbers

```
#!/bin/bash
read -p "Enter a: " a
read -p "Enter b: " b
read -p "Enter c: " c
if [ $a -ge $b ] && [ $a -ge $c ]; then echo "$a is greatest"
elif [ $b -ge $a ] && [ $b -ge $c ]; then echo "$b is greatest"
else echo "$c is greatest"
fi
```

11) Factorial

```
#!/bin/bash
read -p "Enter n: " n
fact=1
```

```

for ((i=2; i<=n; i++)); do
    fact=$((fact * i))
done
echo "Factorial of $n = $fact"

```

12) Sum up to N

```

#!/bin/bash
read -p "Enter n: " n
sum=0
for ((i=1; i<=n; i++)); do
    sum=$((sum + i))
done
echo "Sum = $sum"

```

13) Palindrome Number

```

#!/bin/bash
read -p "Enter a number: " num
rev=0; temp=$num
while [ $temp -ne 0 ]; do
    rev=$((rev*10 + temp%10))
    temp=$((temp/10))
done
if [ $num -eq $rev ]; then echo "Palindrome"
else echo "Not Palindrome"
fi

```

14) Prime Number

```

#!/bin/bash
read -p "Enter number: " n
flag=0
for ((i=2; i<=n/2; i++)); do
    if [ $((n%i)) -eq 0 ]; then flag=1; break; fi
done
if [ $flag -eq 0 ] && [ $n -gt 1 ]; then echo "Prime"
else echo "Not Prime"
fi

```

15) Fibonacci Series

```

#!/bin/bash
read -p "How many terms? " n
a=0; b=1
echo -n "$a $b "
for ((i=2; i<n; i++)); do
    c=$((a+b)); echo -n "$c "; a=$b; b=$c
done
echo

```

16) Max & Min in Array

```
#!/bin/bash
arr=(3 1 7 2 9 4)
max=${arr[0]}; min=${arr[0]}
for x in "${arr[@]"; do
    [ $x -gt $max ] && max=$x
    [ $x -lt $min ] && min=$x
done
echo "Max=$max Min=$min"
```

17) Linear Search

```
#!/bin/bash
arr=(10 20 30 40 50)
read -p "Search: " key
found=0
for ((i=0;i<${#arr[@]};i++)); do
    if [ ${arr[$i]} -eq $key ]; then
        echo "Found at index $i"; found=1; break
    fi
done
[ $found -eq 0 ] && echo "Not found"
```

18) Bubble Sort

```
#!/bin/bash
arr=(64 34 25 12 22 11 90)
n=${#arr[@]}
for ((i=0;i<n-1;i++)); do
    for ((j=0;j<n-i-1;j++)); do
        if [ ${arr[$j]} -gt ${arr[$((j+1))]} ]; then
            tmp=${arr[$j]}; arr[$j]=${arr[$((j+1))]}; arr[$((j+1))]=$tmp
        fi
    done
done
echo "Sorted: ${arr[@]}"
```

19) Matrix Multiplication

```
#!/bin/bash
# 2x2 matrix multiplication
A=(1 2 3 4); B=(5 6 7 8)
for ((i=0;i<2;i++)); do
    for ((j=0;j<2;j++)); do
        sum=0
        for ((k=0;k<2;k++)); do
            sum=$((sum + A[i*2+k]*B[k*2+j]))
        done
        echo -n "$sum "
    done
done
echo
```

20) Sum of All Elements of Array

```
#!/bin/bash
arr=(1 2 3 4 5)
sum=0
for x in "${arr[@]}"; do sum=$((sum+x)); done
echo "Sum = $sum"
```

21) PYTHON (marker in index – see Python section below)

```
# See Python programs below (programs 22 onwards are in Python)
```

■■■ Python Programs ■■■

22) Add 2 Numbers

```
a = int(input("Enter a: "))
b = int(input("Enter b: "))
print("Sum =", a + b)
```

23) Average of 5 Numbers

```
nums = [int(input(f"Enter number {i+1}: ")) for i in range(5)]
print("Average =", sum(nums) / 5)
```

24) Sum up to N

```
n = int(input("Enter n: "))
print("Sum =", sum(range(1, n+1)))
```

25) Factorial

```
n = int(input("Enter n: "))
fact = 1
for i in range(2, n+1):
    fact *= i
print(f"Factorial of {n} =", fact)
```

26) Fibonacci Series

```
n = int(input("How many terms? "))
a, b = 0, 1
for _ in range(n):
    print(a, end=" ")
    a, b = b, a + b
print()
```

27) Prime Number or Not

```
n = int(input("Enter number: "))
is_prime = n > 1 and all(n % i != 0 for i in range(2, int(n**0.5)+1))
print("Prime" if is_prime else "Not Prime")
```

28) Maximum in a List

```
lst = list(map(int, input("Enter numbers separated by space: ").split()))
print("Maximum =", max(lst))
```

29) Bubble Sort

```
lst = list(map(int, input("Enter numbers: ").split()))
n = len(lst)
```

```

for i in range(n-1):
    for j in range(n-i-1):
        if lst[j] > lst[j+1]:
            lst[j], lst[j+1] = lst[j+1], lst[j]
print("Sorted:", lst)

```

30) Searching in a List

```

lst = list(map(int, input("Enter list: ").split()))
key = int(input("Search key: "))
if key in lst:
    print(f"Found at index {lst.index(key)}")
else:
    print("Not found")

```

31) Addition of All Elements of List

```

lst = list(map(int, input("Enter list: ").split()))
print("Sum =", sum(lst))

```

32) Matrix Multiplication

```

def mat_mul(A, B):
    rows_A, cols_A = len(A), len(A[0])
    cols_B = len(B[0])
    C = [[0]*cols_B for _ in range(rows_A)]
    for i in range(rows_A):
        for j in range(cols_B):
            for k in range(cols_A):
                C[i][j] += A[i][k] * B[k][j]
    return C

A = [[1,2],[3,4]]
B = [[5,6],[7,8]]
result = mat_mul(A, B)
for row in result:
    print(row)

```


■■■ Python OOP Programs ■■■

33) PYTHON OOP – Add 2 Complex Numbers + Operator Overloading

```
class Complex:
    def __init__(self, r, i):
        self.r = r
        self.i = i

    def __add__(self, other):
        return Complex(self.r + other.r, self.i + other.i)

    def __str__(self):
        return f"{self.r} + {self.i}i"

c1 = Complex(3, 4)
c2 = Complex(1, 2)
print("Sum:", c1 + c2)
```

34) Area of Shapes → Polymorphism

```
import math

class Shape:
    def area(self):
        return 0

class Circle(Shape):
    def __init__(self, r): self.r = r
    def area(self): return math.pi * self.r ** 2

class Rectangle(Shape):
    def __init__(self, l, b): self.l = l; self.b = b
    def area(self): return self.l * self.b

class Triangle(Shape):
    def __init__(self, b, h): self.b = b; self.h = h
    def area(self): return 0.5 * self.b * self.h

shapes = [Circle(5), Rectangle(4, 6), Triangle(3, 8)]
for s in shapes:
    print(f"{s.__class__.__name__} area = {s.area():.2f}")
```

35) Inheritance + String Concatenation of 2 Strings

```
# Inheritance
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        return "..."

class Dog(Animal):
```

```

def speak(self):
    return f"{self.name} says Woof!"

class Cat(Animal):
    def speak(self):
        return f"{self.name} says Meow!"

d = Dog("Rex")
c = Cat("Whiskers")
print(d.speak())
print(c.speak())

# String Concatenation - simple, multiple, multilevel
s1 = "Hello"
s2 = " World"
print("Simple concat:", s1 + s2)

# Multiple
parts = ["Python", " is", " fun"]
print("Multiple concat:", "".join(parts))

# Multilevel (nested join)
level1 = "foo" + "bar"
level2 = level1 + "baz"
print("Multilevel:", level2)

```

— End of All Programs —